Adelaide University

Microcontroller Programming

Practical 6 Manual

FSM, Ultrasonic sensor and Kobuki bytestream

NUCLEO-G474RE • FSM •Ultrasonic sensor • Kobuki bytestream

Student Edition

Practical outcome: Build separate ADC and ultrasonic modules,measure the MB1043 analogue output on PA0, identify an obstacle within 700 mm, and integrate FSM result with the existing Kobuki motor controller.

Contents

1 Practical Overview 1

2 Hardware 1

3 Part A – Wire the MB1043 Sensor 1

4 Part B – Files Architecture 3

5 Part C – Create the ADC Driver 4

6 Part D – Create the Ultrasonic Driver 6

7 Part E – Integrate the Sensor in `main.c` 7

8 Part F – Finite State Machine for Robot Control 10

1. Practical Overview

In Practical 5, you configured USART3 and wrote the UART and Kobuki device drivers. In this practical, retain that working project and extend it with an MB1043 HRLV-MaxSonar-EZ4 ultrasonic sensor. The sensor measures distance and provides analogue, pulse-width and serial outputs. We will use the analogue output on Pin 3, connected to STM32G474 pin PA0 (ADC1 input channel 1).

As in Practical 5, low-level microcontroller code is kept separate from the higher-level device driver. The ADC driver configures and reads ADC1. The ultrasonic driver converts the ADC result into millimetres and reports whether an object is detected. The program in `main.c` combines this result with the Kobuki button, bumper and cliff data through a finite-state machine (FSM).

For reference, you will need the STM32G474 reference manual, NUCLEO-G474RE user manual, MB1043 datasheet, Kobuki user guide and Kobuki serial bytestream document. Keep your completed Practical 5 project available throughout.

2. Hardware

- STM32G474 development board;
- Kobuki mobile robot and its appropriate UART interface cable;
- MB1043 HRLV-MaxSonar-EZ4 ultrasonic sensor;
- Jumper wires and a USB programming cable;
- Visual Studio Code with STM32Cube extensions; and
- the existing Kobuki project containing `uart.c`, `uart.h`, `kobuki.c` and `kobuki.h`.

Raise the Kobuki wheels clear of the bench during the first motor test. Do not connect STM32 GPIO directly to a true voltage-level signal.

Use the NUCLEO-G474RE and Kobuki assembly completed in Practical 5. Confirm that the mounting plate is secure and that the UART cable remains connected (kobuki_RX to PC10, kobuki_TX to PC11, kobuki_GND to Pin8 GND) before adding the ultrasonic sensor.

3. Part A – Wire the MB1043 Sensor

Task 1: Identify the sensor pins

Connect the sensor in analogue free-running mode.

MB1043 pin	Signal	STM32G474 connection	Purpose
Pin 1	Temperature	Leave unconnected	Optional external compensation
Pin 2	Pulse width	Leave unconnected	Not used in this practical
Pin 3	AN output	PA0 / ADC1_IN1 / A0	Analogue distance signal
Pin 4	RX control	Leave unconnected	Internal pull-up enables free run
Pin 5	Serial output	Leave unconnected	Not used in this practical
Pin 6	V+	3V3 (Pin 16)	Sensor power
Pin 7	GND	GND (Pin 20)	Common reference

Essential wiring: Pin 6 to 3V3, Pin 7 to GND and Pin 3 to PA0/A0. The STM32, MB1043 and Kobuki must share a common signal ground.

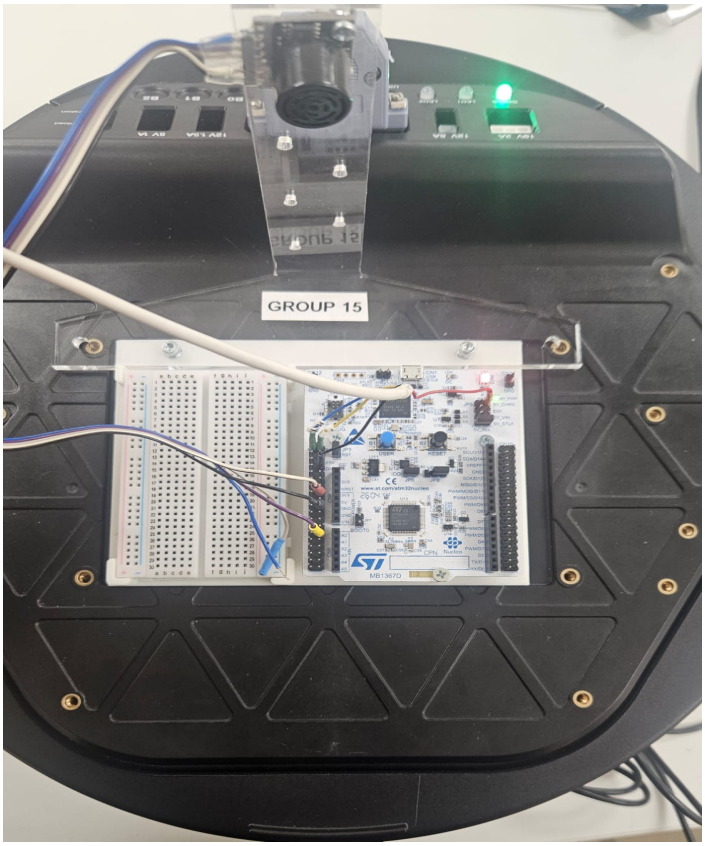


Figure 1: Kobuki robot with the NUCLEO-G474RE mounted (continued from Practical 5).

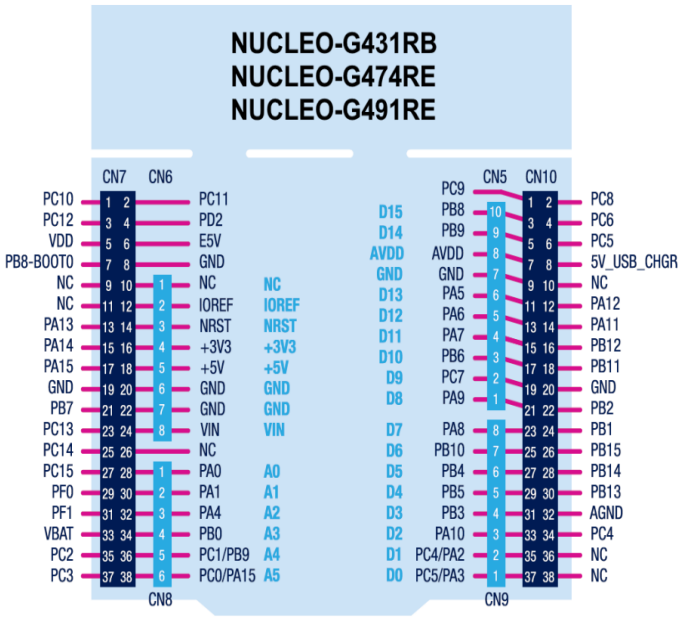


Figure 2: NUCLEO-G474RE connector layout used to locate PA0/A0.

4. Part B – Files Architecture

Task 2: Creating File structure in Src

Do not place the ADC and ultrasonic wrappers directly into `main.c`. Following the structure introduced in Practical 5, create `ADC` and `Ultrasonic` inside `Src`.

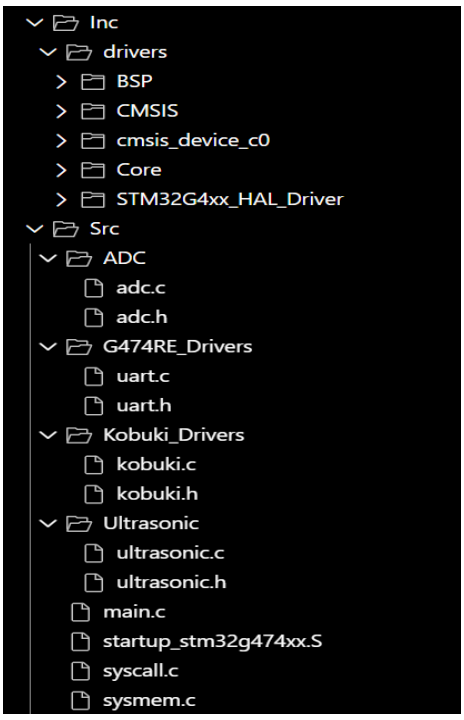


Figure 3: File layout in Src folder.

File	Responsibility
`adc.h`	ADC public constants and function declarations
`adc.c`	ADC1 clock, calibration, enabling and channel conversion
`ultrasonic.h`	Sensor data structure, threshold and declarations
`ultrasonic.c`	PA0 configuration, distance conversion and detection
`main.c`	FSM, Kobuki input processing and motor commands

Your existing `G474RE_Drivers` and `Kobuki_Drivers` folders from Practical 5 must remain in the project. The completed structure should contain all four driver folders before you update `CMakeLists.txt`.

5. Part C – Create the ADC Driver

Task 3: Create `adc.h`

Create `adc.h` in the Src/ADC directory. It exposes the ADC channel and two public functions.

```
#ifndef ADC_H
#define ADC_H

#include <stdint.h>

#define ADC_CHANNEL_1  ((uint8_t)1U)

void ADC_Init(void);
uint16_t ADC_Read(uint8_t channel);

#endif
```

Task 4: Start `adc.c`

Create `adc.c` and include the STM32G474 device and driver headers.

```
#include <stdint.h>
#define STM32G474xx
#include "stm32g4xx.h"
#include "adc.h"
```

Task 5: Initialise ADC1

Complete the missing register expressions.

```
void ADC_Init(void)
{
/* Enable the ADC12 peripheral clock. */
    RCC->AHB2ENR |= ____________________________;
```

```
/* Select synchronous HCLK/1 as the ADC clock. */

    ADC12_COMMON->CCR &= ~ADC_CCR_CKMODE;
    ADC12_COMMON->CCR |= ADC_CCR_CKMODE_0;

/* Exit deep-power-down mode and enable the ADC voltage regulator
   . */
    ADC1->CR &= ~ADC_CR_DEEPPWD;
    ADC1->CR |= ADC_CR_ADVREGEN;

/* Allow the voltage regulator to stabilise. */
    for (volatile uint32_t i = 0U; i < 1000U; ++i)
        __NOP();

/* Calibrate ADC1 in single-ended mode. */
    ADC1->CR &= ~ADC_CR_ADCALDIF;
    ADC1->CR |= ____________________________;
    while ((ADC1->CR & ADC_CR_ADCAL) != 0U) { }

/* Enable ADC1 and wait until it is ready. */

    ADC1->ISR = ADC_ISR_ADRDY;
    ADC1->CR |= ____________________________;
    while ((ADC1->ISR & ADC_ISR_ADRDY) == 0U) { }
}
```

Task 6: Read an ADC channel

The function selects one regular channel, starts a conversion, waits for the end-of-conversion flag and returns the 12-bit result.

```
uint16_t ADC_Read(uint8_t channel)
{
    /* This driver supports ADC1 regular channels 1 to 18. */
    if ((channel < 1U) || (channel > 18U))
        return 0U;
/* Select one regular conversion and place the channel in SQ1. */

    ADC1->SQR1 = ((uint32_t)channel << ADC_SQR1_SQ1_Pos);

/* Select the longest sample time for a stable sensor measurement
   . */
    if (channel <= 9U) {
        uint32_t position = (uint32_t)channel * 3U;
        ADC1->SMPR1 &= ~(7UL << position);
        ADC1->SMPR1 |=  (7UL << position);
    }
    else {
        uint32_t position = ((uint32_t)channel - 10U) * 3U;
        ADC1->SMPR2 &= ~(7UL << position);
        ADC1->SMPR2 |=  (7UL << position);
    }
```

```
    /* Clear completion flags and start the conversion. */
    ADC1->ISR = ADC_ISR_EOC | ADC_ISR_EOS;
    ADC1->CR |= ____________________________;

    while ((ADC1->ISR & ____________________________) == 0U) {
    }

    return (uint16_t)(ADC1->DR & ADC_DR_RDATA);
}
```

6. Part D – Create the Ultrasonic Driver

Task 7: Create `ultrasonic.h`

```
#ifndef ULTRASONIC_H
#define ULTRASONIC_H

#include <stdint.h>

#define ULTRASONIC_DETECT_RANGE_MM ((uint16_t)700U)
#define ULTRASONIC_MAX_RANGE_MM    ((uint16_t)5000U)

typedef struct {
    uint8_t detected;
    uint16_t range_mm;
} US_Typedef;

void Ultrasonic_Init(void);
uint16_t Ultrasonic_Read(void);
void Ultrasonic_Detect(US_Typedef *object);

#endif
```

Task 8: Configure PA0 in `ultrasonic.c`

```
#include <stdint.h>
#define STM32G474xx
#include "stm32g4xx.h"
#include "adc.h"
#include "ultrasonic.h"

#define ULTRASONIC_CHANNEL   ADC_CHANNEL_1
#define ADC_MAX_VALUE        4095UL
#define MB1043_FULL_SCALE_MM 5120UL

void Ultrasonic_Init(void)
{
    RCC->AHB2ENR |= ____________________________;
```

```
    GPIOA->MODER |= GPIO_MODER_MODE0;
    GPIOA->PUPDR &= ~GPIO_PUPDR_PUPD0;
}
```

`ADC_Init()` configures the ADC peripheral. In contrast, `Ultrasonic_Init()` configures the sensor-specific PA0 input pin.

Task 9: Convert the ADC result

```
uint16_t Ultrasonic_Read(void)
{
    uint16_t adc_value = ADC_Read(ULTRASONIC_CHANNEL);

    uint32_t distance_mm =
        ((uint32_t)adc_value * ________________ +
         (ADC_MAX_VALUE / 2UL)) / ________________;

    if (distance_mm > ULTRASONIC_MAX_RANGE_MM)
        distance_mm = ULTRASONIC_MAX_RANGE_MM;

    return (uint16_t)distance_mm;
}
```

Task 10: Determine whether an obstacle is present

```
void Ultrasonic_Detect(US_Typedef *object)
{
    if (object == 0)
        return;

    object->range_mm = ________________________;
    object->detected =
        (object->range_mm < ________________________)
        ? 1U : 0U;
}
```

7. Part E – Integrate the Sensor in `main.c`

Task 11: Include and initialise the modules

Add the new headers and initialise the drivers once before the control loop.

```
#include <stdint.h>

#define STM32G474xx
#include "stm32g4xx.h"
#include "adc.h"  // New Header
#include "kobuki.h"
```

```c
#include "uart.h"
#include "ultrasonic.h" // New Header

#define BUTTON_0_START   0x01U //Confirm the values from
   my_kobuki
#define BUTTON_1_STOP    0x02U //Confirm the values from
   my_kobuki
#define BUTTON_2_ROTATE  0x04U //Confirm the values from
   my_kobuki

#define DRIVE_SPEED_MM_S        ((int16_t)100)
#define ROTATE_SPEED_MM_S       ((int16_t)100)

static void Send_Kobuki_Move(int16_t speed, int16_t radius)
{
    uint8_t payload[6];

    payload[0] = 0x01U; /* Base Control sub-packet ID. */
    payload[1] = 0x04U; /* Speed and radius occupy four bytes. */

    payload[2] = (uint8_t)(speed & 0xFF);
    payload[3] = (uint8_t)((uint16_t)speed >> 8);
    payload[4] = (uint8_t)(radius & 0xFF);
    payload[5] = (uint8_t)((uint16_t)radius >> 8);

    Kobuki_Tx(payload, sizeof(payload));
}

int main(void)
{
    Kobuki_Typedef my_kobuki = {0};
    US_Typedef ultrasonic_object = {0};
    uint8_t is_running = 0U;
    uint8_t is_rotating = 0U;

    UART_Init();
    ADC_Init();
    Ultrasonic_Init();
    Send_Kobuki_Move(0, 0);

    while (1) {
        /* Control loop completed in Task 12. */
    }
}
```

Task 12: Complete the control loop

The required behaviour is:

- Button 0 selects forward motion when no obstacle is detected;
- Button 2 selects rotation when no obstacle is detected;

- Button 1, the bumper or an ultrasonic obstacle stops the robot; and
- the detection distance is < 700 mm.

```
while (1)
{
    Kobuki_Read(&my_kobuki);
    Ultrasonic_Detect(&ultrasonic_object);

/* Button 0 selects forward motion when the path is clear. */

    if (((my_kobuki.button & ________) != 0U) &&
        (ultrasonic_object.detected == 0U)) {
        is_running = 1U;
        is_rotating = 0U;
    }

     /* Button 2 selects rotation when the path is clear. */

    else if (((my_kobuki.button & ________) != 0U) &&
             (ultrasonic_object.detected == 0U)) {
        is_rotating = 1U;
        is_running = 0U;
    }

    /* Button 1, bumper or ultrasonic detection stops the robot.
   */

    if (((my_kobuki.button & _________) != 0U) ||
        (my_kobuki.bumper != 0U) ||
        (ultrasonic_object.detected != 0U)) {
        is_running = 0U;
        is_rotating = 0U;
    }

    if (is_running != 0U)
        Kobuki_Drive(100);
    else if (is_rotating != 0U)
        Kobuki_Rotate(100);
    else
        Send_Kobuki_Move(0, 0);
}
```

Task 13: Update the project configuration

Add both implementation files to the CMake target sources:

```
target_sources(${PROJECT_NAME} PRIVATE
    Src/G474RE_Drivers/uart.c
    Src/Kobuki_Drivers/kobuki.c
    Src/ADC/adc.c
    Src/Ultrasonic/ultrasonic.c
```

```
)
```

Add their header directories:

```
target_include_directories(${PROJECT_NAME} PRIVATE
 // # Retain other driver directories from Practical 5
    Src/G474RE_Drivers
    Src/Kobuki_Drivers
    Src/ADC
    Src/Ultrasonic
)
```

Press `F7`, or run **CMake: Build**. Resolve all compiler and linker errors before connecting the motors.

8. Part F – Finite State Machine for Robot Control

The Finite State Machine (FSM) controls the movement of the Kobuki robot using the push buttons, ultrasonic sensor, bumper sensors, and cliff sensors. The ultrasonic sensor supplies the Boolean input `ultrasonic_object.detected`; the Kobuki supplies button, bumper and cliff. The selected FSM state determines the motor command sent to the robot.

Part F replaces the temporary button-controlled program developed in Part E. In the final FSM, Button 0 starts autonomous roaming, Button 1 stops the robot, and Button 2 is not used. Keep the working UART, ADC and ultrasonic initialisation code.

Task 14: Understand the FSM before completing the code

The program uses five states. Each state has one purpose and one motor action.

State	Purpose	Motor action
`IDLE`	Safe stopped condition	`Send_Kobuki_Move(0,0)`
`SEARCH`	Search for an object	Rotate at 75 mm/s
`OBJECT`	Approach the detected object	Drive forward at 100 mm/s
`OBSTACLE`	Move away from a bumper or cliff event	Drive backward at 100 mm/s
`AVOID`	Turn away from the obstacle	Rotate at 100 mm/s during the avoidance delay

An FSM changes state only when a transition condition becomes true. If no condition is true, the robot remains in its current state and repeats that state's motor action.

Current state	Transition condition	Next state
Initialisation	Program starts	`IDLE`
`IDLE`	Button 0 is pressed	`SEARCH`
`SEARCH`	Ultrasonic object detection is active	`OBJECT`
`OBJECT`	Bumper **or** cliff input is active	`OBSTACLE`
`OBJECT`	Ultrasonic object detection becomes clear	`SEARCH`
`OBSTACLE`	Bumper **and** cliff inputs are both clear	`AVOID`
`AVOID`	Avoidance rotation and delay are completed	`SEARCH`
Any active state	Button 1 is pressed	`IDLE`

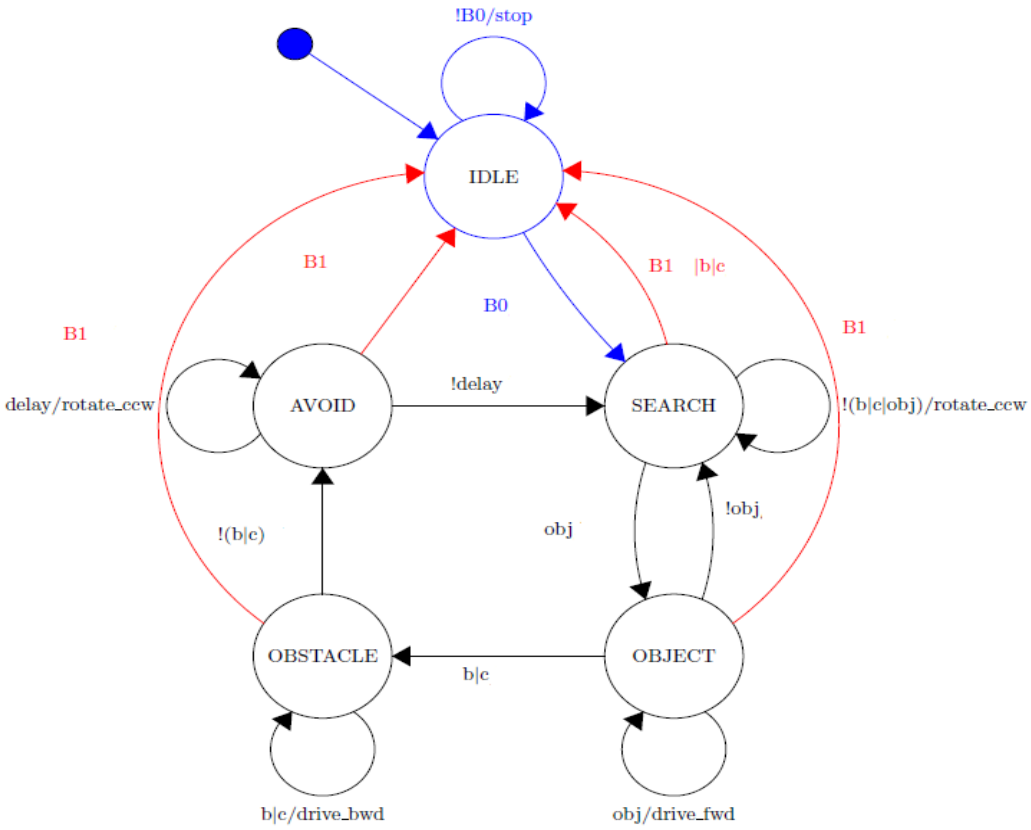


Figure 4: FSM State Transition Diagram

The above state diagram represents the robot's roaming function. The symbol `b` represents the bumper sensor, `c` represents the cliff sensor, `!` denotes logical NOT, and `|` denotes logical OR. The labels `B0` and `B1` represent Buttons 0 and 1, respectively. State-transition conditions and the corresponding robot actions are separated using the `/` symbol. The red arrows show the transitions back to the `IDLE` state. The blue arrows represent the initial transition and the path from `IDLE` to the active states. The black arrows represent the object-searching and obstacle-avoidance functionality.

How the FSM Code Executes

During each iteration of the `while(1)` loop, the program performs the following sequence:

1. `Kobuki_Read()` updates the button, bumper and cliff data.
2. `Ultrasonic_Detect()` updates the object-detection result.
3. Button 1 is checked first as the global stop command.
4. The first `switch` statement evaluates the transition conditions and updates `state`.
5. The second `switch` statement executes exactly one motor action for the selected state.
6. In the `AVOID` state, the robot rotates, waits for the avoidance delay, and then sets the next state to `SEARCH`.

For example, when the robot is in `SEARCH` and the ultrasonic sensor detects an object, the first `switch` changes the state to `OBJECT`. The second `switch` then executes the forward-drive command associated with `OBJECT`.

Task 15: Replace the initialisation and control loop

Add the following declarations above `main()`:

```
/* Kobuki button values */
#define BUTTON_0_START          0x01U
#define BUTTON_1_STOP           0x02U

/* Motor speeds in mm/s */
#define SEARCH_SPEED_MM_S       ((int16_t)75)
#define FORWARD_SPEED_MM_S      ((int16_t)100)
#define REVERSE_SPEED_MM_S      ((int16_t)-100)
#define AVOID_SPEED_MM_S        ((int16_t)100)

/* Approximate delay for the AVOID state.
 * Adjust this value if the rotation is too short or too long.*/
#define AVOID_DELAY_COUNT       300000U

typedef enum {
    IDLE = 0,
    SEARCH,
    OBJECT,
    OBSTACLE,
    AVOID
} RobotState;
static void AvoidDelay(void)
{
    for (volatile uint32_t i = 0U;
         i < AVOID_DELAY_COUNT;
         ++i) {
        __NOP();
    }
```

```
}
```

Use the following initialisation and control-loop structure. Complete the marked blanks using the transition table. Do not change the order of the conditions in the `OBJECT` state.

```
int main(void)
{
    Kobuki_Typedef my_kobuki = {0};
    US_Typedef ultrasonic_object = {0};

    RobotState state = IDLE;

    /* Initialise the drivers. Keep this sequence because it is
   the same as your correctly working ultrasonic program.*/
    UART_Init();
    ADC_Init();
    Ultrasonic_Init();

    /* Start in the safe stopped condition. */
    Send_Kobuki_Move(0, 0);

    while (1) {

        Kobuki_Read(&my_kobuki);
        Ultrasonic_Detect(&ultrasonic_object);

    /* Button 1 is the global stop command. It has priority over
   every other state and input.*/
        if ((my_kobuki.button & BUTTON_1_STOP) != 0U) {
            state = IDLE;
        }
        else {
            //FSM transition logic
            switch (state) {

            case IDLE:
//The robot remains stopped until Button 0 is pressed.
                if ((my_kobuki.button &
                     BUTTON_0_START) != 0U) {

                    state = SEARCH;
                }
                break;

            case SEARCH:
/* The robot rotates while searching. When the ultrasonic sensor
   detects an object, change to OBJECT.*/
                if (ultrasonic_object.detected != 0U) {
                    state = OBJECT;
                }
            break;
```

Complete the transition blanks

The first condition below detects a physical hazard. The second condition checks whether the ultrasonic object has been lost. In `OBSTACLE`, the robot may leave the state only when both safety inputs are clear.

```
            case OBJECT:
                /* The robot drives towards the object. A bumper
   or cliff input changes the state to OBSTACLE.
                 */
                if ((my_kobuki.bumper ___) ||
                    (my_kobuki.cliff ____)) {

                    state = _____;
                }
                /*If the ultrasonic sensor loses the object,
   return to SEARCH.*/
                else if (ultrasonic_object.detected ____) {
                    state = _____;
                }
                break;

            case OBSTACLE:
                /* The robot reverses while the bumper or
                 * cliff input remains active. When both inputs
   clear, change to AVOID.*/
                if ((my_kobuki.bumper ____) &&
                    (my_kobuki.cliff _____)) {

                    state = ____;
                }
                break;

            case AVOID:
                //The timed avoidance movement is applied in the
   motor-output switch below.
                     break;

            default:
                /* * Recover safely if the state variable
   contains an unexpected value. */
                state = IDLE;
                break;
            }
        }
```

Before filling a blank, first identify whether the input must be active or clear, and then identify the required next state. A bumper or cliff hazard leads to OBSTACLE; losing the object leads to SEARCH; clearing both safety inputs leads to AVOID.

Apply one motor action for the selected state

The output switch does not decide transitions. It only executes the action belonging to the state selected above.

```
        /*FSM output actions Exactly one motor command is
   selected according to the current state.
         */
        switch (state) {

        case IDLE:
            //Stop the robot.
            Send_Kobuki_Move(0, 0);
            break;

        case SEARCH:
            // Rotate counter-clockwise while searching.
            Kobuki_Rotate(SEARCH_SPEED_MM_S);
            break;

        case OBJECT:
            // Drive towards the detected object.
            Kobuki_Drive(FORWARD_SPEED_MM_S);
            break;

        case OBSTACLE:
            // Reverse after bumper or cliff detection.
            Kobuki_Drive(REVERSE_SPEED_MM_S);
            break;

        case AVOID:
            //Rotate to avoid the obstacle and then return to the
    SEARCH state.

            Kobuki_Rotate(AVOID_SPEED_MM_S);
            AvoidDelay();
            state = SEARCH;
            break;

        default:
            // Safe fallback.
            Send_Kobuki_Move(0, 0);
            state = IDLE;
            break;
        }

        //Small control-loop delay.
        for (volatile uint32_t i = 0U;
             i < 5000U;
             ++i) {
            __NOP();
        }
```

```
    }
}
```

For the initial trial: Before running the program on the robot, place the Kobuki on a small block to prevent it from running off the table. Kobuki wheels cannot touch the table. Run and Debug your program and test the code execution.
Once you are happy that the code runs properly, you can test it on the ground, beware of the surroundings when doing so.